

# Recueil complet

*RL multi-agents pour agents de combat — projet Arma 3*

*écosystème + 8 algorithmes + l'art du RL*

# Comprendre les algorithmes du RL multi-agents

## Cours en clair — du débutant à la maîtrise, sans être mathématicien

Ce recueil est conçu comme un **cours**. Chaque fiche suit la même progression pédagogique :

1. **Le problème** — la difficulté précise que l'algorithme attaque, et pourquoi les approches naïves échouent.
2. **L'idée centrale** — l'intuition qui débloque tout, en une phrase.
3. **Le mécanisme, pas à pas** — comment ça marche réellement, étape par étape.
4. **Les formules, traduites** — chaque formule est donnée, dite en mots, puis illustrée par une image « *pour un littéraire* ». On explique aussi *pourquoi* chaque terme est là.
5. **Un exemple concret** — l'algorithme à l'œuvre dans une scène de combat.
6. **Variantes & pièges** — les cousins de l'algorithme, et ses modes d'échec connus.
7. **Pourquoi on l'utilise ici** — son rôle dans ton projet Arma 3.
8. **En dernier regard** — une lecture *philosophique*.

Aucune formule n'est laissée nue : elle est toujours traduite. Tu peux tout comprendre sans mathématiques — mais le contenu, lui, est de niveau expert.

## Ordre de lecture

| #   | Fiche                                     | La question qu'elle résout                                                   |
|-----|-------------------------------------------|------------------------------------------------------------------------------|
| 00b | <b>Écosystème du RL</b>                   | D'où vient tout ça, et comment le domaine s'organise-t-il ?                  |
| 01  | <b>PPO / MAPPO</b>                        | Comment progresser sans que l'apprentissage explose ?                        |
| 02  | <b>HAPPO</b>                              | Comment des agents <i>différents</i> s'améliorent-ils ensemble, à coup sûr ? |
| 03  | <b>QMIX &amp; décomposition de valeur</b> | Comment des décisions locales servent-elles le bien de l'équipe ?            |
| 04  | <b>COMA</b>                               | Qui, dans l'équipe, a <i>vraiment</i> mérité le succès ?                     |
| 05  | <b>CMDP / Lagrangien</b>                  | Comment réussir la mission <i>sans</i> dépasser un seuil de pertes ?         |
| 06  | <b>Auto-jeu &amp; PSRO / ligue</b>        | Comment forger des agents robustes face à un adversaire intelligent ?        |
| 07  | <b>PBT</b>                                | Comment régler et diversifier les entraînements tout seuls ?                 |
| 08  | <b>PLR (curriculum)</b>                   | Comment apprendre à généraliser plutôt qu'à mémoriser ?                      |

**Deux familles.** Les fiches **00b** → **05** décrivent *comment une équipe apprend* (la « boucle interne »). Les fiches **06** → **08** décrivent *comment on organise les entraînements entre eux* (la « boucle externe »). À lire en complément : le dossier « **Le RL — maîtriser l'art** » (forces, échecs, remèdes).

# Écosystème du RL

« *D'où vient tout ça, et comment le domaine s'organise* »

## Le problème qu'il résout (de quoi parle-t-on ?)

Avant de plonger dans les algorithmes, il faut une **carte**. Sans elle, ils semblent un fatras de sigles. En réalité, ils descendent tous d'une même souche et répondent tous à une seule question : *comment un agent apprend-il un bon comportement à partir d'une simple récompense ?*

## L'idée centrale : trois familles d'apprentissage

L'apprentissage automatique se divise en trois grandes familles. Comprendre où se situe le RL, c'est comprendre sa nature profonde.

| Famille                  | Ce qu'on lui donne                                                     | Ce qu'elle apprend                            |
|--------------------------|------------------------------------------------------------------------|-----------------------------------------------|
| <b>Supervisé</b>         | des exemples <i>étiquetés</i> (« ceci est un char »)                   | à imiter les bonnes réponses                  |
| <b>Non supervisé</b>     | des données <i>sans</i> étiquettes                                     | à y trouver des structures cachées            |
| <b>Renforcement (RL)</b> | <i>aucune</i> bonne réponse — seulement une <b>récompense</b> différée | à <b>agir</b> pour maximiser cette récompense |

*Pour un littéraire* : le supervisé est un élève à qui l'on dicte les corrigés ; le RL est un enfant lâché dans le monde, qui n'apprend que des conséquences de ses actes. C'est plus lent, plus capricieux — mais c'est la seule façon d'apprendre à *agir* quand personne ne connaît d'avance la bonne conduite.

## Le mécanisme, pas à pas : la boucle fondamentale

Tout le RL tient dans une boucle, répétée des millions de fois :

I'AGENT observe → choisit une ACTION → l'ENVIRONNEMENT change et renvoie une RÉCOMPENSE → l'agent ajuste sa politique → on recommence

1. L'agent perçoit l'état (ou une fraction de l'état).
2. Sa **politique** (sa façon de décider) choisit une action.
3. L'environnement répond : nouvel état + récompense.
4. L'agent met à jour sa politique pour que les actions « payantes » deviennent plus probables.

Tout le reste — PPO, QMIX, COMA... — n'est qu'une **manière différente de faire l'étape 4** (ajuster la politique) ou d'**organiser la boucle** (contre qui, sur quoi).

## La généalogie : d'où viennent tes algorithmes

Le RL a deux âges. **L'âge théorique** (années 1950-1990) : Bellman et les *processus de décision markoviens* (la formalisation « état → action → récompense »), la programmation dynamique, le **Q-learning** (Watkins, 1989), et la « bible » du domaine, le livre de **Sutton & Barto**. Puis **l'âge profond** (depuis 2013) : la rencontre du RL avec les **réseaux de neurones**, qui a tout fait exploser, portée par une culture singulière — *les jeux comme bancs d'essai*.

| Algorithme / idée                  | Né chez                                          | Vers    |
|------------------------------------|--------------------------------------------------|---------|
| DQN (le déclic « deep RL », Atari) | DeepMind                                         | 2013-15 |
| PPO, MADDPG                        | OpenAI (+ Berkeley)                              | 2017    |
| VDN, PSRO, PBT, AlphaStar/ligue    | <b>DeepMind</b>                                  | 2017-19 |
| QMIX, COMA, banc d'essai SMAC      | <b>Oxford</b> (labo WhiRL)                       | 2018    |
| MAPPO                              | Tsinghua / Berkeley                              | 2021    |
| HAPPO                              | UCL / Oxford                                     | 2022    |
| PLR / curriculum automatique       | UCL / Meta AI                                    | 2020-21 |
| CMDP (contraintes)                 | théorie du contrôle (1999) → RL (Berkeley, 2017) | —       |

À retenir : une poignée de laboratoires (**DeepMind, OpenAI, Oxford**) a produit l'essentiel, en ~10 ans, autour des jeux (Atari → go → StarCraft → Dota). Ton projet n'est donc pas un bricolage : il s'inscrit dans une lignée **récente, concentrée et cohérente**.

## La carte du domaine (pour t'y retrouver)

- **Mono-agent vs multi-agents (MARL)** : un seul cerveau, ou plusieurs qui interagissent. Ton projet est multi-agents — la branche la plus difficile (les autres bougent aussi).
- **Sur-politique vs hors-politique** : apprendre des données qu'on vient de produire (PPO) ou ressasser de vieilles données stockées (Q-learning, QMIX). Le second est plus économe en données.
- **Par valeur vs par politique** : noter les situations (« valeur ») pour en déduire l'action, ou ajuster directement la façon de décider. La plupart des méthodes modernes mêlent les deux (*acteur-critique*).
- **Coopératif / compétitif / mixte** : ton cas est **mixte** — coopératif dans l'équipe, compétitif contre l'ennemi.

## Pourquoi cette carte t'est utile

Elle explique aussi tes **outils logiciels** : EPyMARL, RLlib, OmniSafe, PettingZoo sont nés de cette même communauté pour implémenter exactement ces algorithmes. Théorie, code et bancs d'essai

viennent du même monde — ce qui veut dire que tout ce que tu apprends ici se branche directement sur des bibliothèques existantes.

*En dernier regard. Le RL est, de toutes les approches de l'intelligence artificielle, la plus proche de la condition vivante. Le supervisé apprend dans une bibliothèque, entouré de corrigés ; le RL apprend dans l'arène, où nul ne lui dit la vérité, où seule compte la conséquence. C'est l'empirisme poussé à sa limite — l'idée, déjà chère à Hume puis aux pragmatistes, que nous ne connaissons le monde qu'en agissant sur lui et en éprouvant le retour de nos actes. Qu'une discipline d'ingénieurs ait redécouvert, en voulant faire jouer des machines, la vieille question de savoir comment l'expérience devient sagesse, n'est pas le moindre de ses charmes.*

# PPO / MAPPO

---

« *Progresser à petits pas garantis* »

## Le problème qu'il résout

---

Pour améliorer une politique, l'idée naturelle est : « pousse un peu plus fort vers les gestes qui ont rapporté ». C'est le *gradient de politique*. Mais brut, il a deux défauts mortels. D'abord, il est **très bruité** : on estime « ce qui rapporte » à partir de quelques parties au hasard, donc l'estimation tremble. Ensuite, et surtout, **un seul pas trop grand détruit tout** : la politique bascule dans une région catastrophique, n'y récolte plus que de mauvaises données, et ne s'en relève jamais. En RL, on ne peut pas « annuler » un mauvais pas — les données suivantes sont déjà produites par la politique abîmée.

## L'idée centrale

---

**Améliorer la politique, mais en s'interdisant de trop s'éloigner de la précédente à chaque pas.** C'est l'idée de *région de confiance* : on ne fait confiance à notre estimation que dans un petit voisinage. PPO impose cette retenue d'une manière simple et robuste — en *coupant* la récompense dès qu'on change trop.

## Le mécanisme, pas à pas

---

1. On joue un paquet de parties avec la politique **actuelle** (on récolte états, actions, récompenses).
2. On estime, pour chaque geste, son **avantage** : a-t-il fait mieux ou moins bien que prévu ?
3. On fait plusieurs petites passes d'ajustement qui augmentent la probabilité des gestes à bon avantage — **mais bridées** par le mécanisme de coupure.
4. En parallèle, on entraîne un « critique » (la fonction *valeur*) qui apprend à pronostiquer, pour réduire le bruit des estimations.
5. On jette ces données (elles ne valent que pour l'ancienne politique) et on recommence.

## Les formules, traduites

---

### ① La surprise, brique de base.

$$\delta = (\text{récompense reçue}) + \gamma \cdot V(\text{situation suivante}) - V(\text{situation actuelle})$$

$V$  est le **pronostic** d'une situation ;  $\delta$  est l'**écart** entre ce qui arrive et ce qu'on attendait.  $\gamma$  (entre 0 et 1) dit qu'un bien lointain compte un peu moins qu'un bien immédiat.

*Pour un littéraire* : le commentateur qui révise son jugement sur un coup une fois la suite connue.

③ **L'avantage** — somme des surprises (technique nommée GAE) :

avantage =  $\delta$  maintenant + (un peu moins)  $\delta$  ensuite + (encore moins)  $\delta$  après...

*En clair* : le bilan, bon ou mauvais, qu'un geste a réellement déclenché dans la durée.

③ **Le ratio de changement.**

ratio = (envie de la NOUVELLE politique pour ce geste)  $\div$  (envie de l'ANCIENNE)

*Pourquoi ce terme ?* Il mesure de combien on s'écarte de la politique qui a produit les données. C'est lui qu'on va brider.

④ **Le cœur de PPO — l'objectif coupé.**

on récompense : ratio  $\times$  avantage ... MAIS si le ratio sort de  $[1-\varepsilon, 1+\varepsilon]$ , on le COUPE (plus de récompense au-delà)

Si l'avantage est positif, on veut monter le ratio — mais au-delà de  $1+\varepsilon$  (ex. +20 %), la coupure annule le gain : inutile d'aller plus loin. Symétriquement pour un avantage négatif.

*Pour un littéraire* : un **limiteur de vitesse sur l'enthousiasme**.  $\varepsilon$  est la longueur de la laisse. Foncer sur une impression fragile, c'est se planter ; PPO l'interdit par construction.

⑤ **Le sucre en plus.** On ajoute souvent un petit **bonus d'entropie** : une prime à rester un peu imprévisible, pour ne pas se figer trop tôt sur une seule tactique (encourager l'exploration).

## Un exemple concret

Ton groupe tente un **débordement par la droite**. Sur dix répétitions, il réussit deux fois mieux que la moyenne : avantage positif. PPO augmente la propension à déborder — mais d'un cran seulement. Pourquoi pas à fond ? Parce que ces dix parties sont peu nombreuses : peut-être un coup de chance. La laisse  $\varepsilon$  empêche l'équipe de devenir obsédée par le débordement sur une preuve trop maigre.

## Variantes & pièges

- **TRPO** : l'ancêtre, qui impose la même retenue par une contrainte mathématique dure (puissant mais lourd). PPO l'approxime par la coupure — plus simple, presque aussi bon.

- **IPPO** : chaque agent fait son PPO dans son coin (étonnamment solide en pratique).
- **MAPPO / HAPPO** : les versions équipe (ci-dessous).
- *Pièges* : mal régler  $\epsilon$  (trop grand = instable, trop petit = apprentissage lent) ; oublier de normaliser les récompenses ; mal concevoir ce que voit le critique central.

## Ce que MAPPO ajoute (la version équipe)

---

- **Un critique qui voit tout** pendant l'entraînement (l'état global du champ de bataille), alors que chaque soldat **agit avec ses seuls yeux**. C'est le principe CTDE : apprendre avec une vue d'ensemble, exécuter en aveugle partiel.
- **Un cerveau partagé, décliné par rôle** : les agents mutualisent leur expérience.

## Pourquoi on l'utilise ici

---

La prudence intégrée (la laisse  $\epsilon$ ) **dompte l'instabilité du multi-agents** : quand les coéquipiers changent sans cesse, les petits pas bornés évitent l'effondrement. C'est, en pratique, la base la plus robuste et la plus simple à faire tenir debout — le bon point de départ.

*En dernier regard. PPO est une philosophie de la mesure faite algorithme. Il refuse les deux tentations symétriques de l'esprit : l'immobilisme qui n'apprend de rien, et l'enthousiasme qui réforme tout sur une intuition fragile. Aristote nommait vertu ce juste milieu, et prudence (phronèsis) l'art d'avancer sans se renier. PPO formalise une sagesse très ancienne : on ne progresse pas en se reniant à chaque épreuve, mais en corrigeant le cap d'un degré à la fois — assez pour avancer, assez peu pour rester soi-même et pouvoir se corriger encore demain.*

# HAPPO

---

« *S'améliorer ensemble, à coup sûr, quand on est différents* »

## Le problème qu'il résout

---

MAPPO suppose volontiers que les agents sont interchangeables (un même cerveau partagé). Mais ton équipe est faite de **rôles distincts** : un chef, un médecin, un mitrailleur. Surtout, un danger guette toute équipe qui apprend : si **tous se corrigent en même temps**, chacun suppose que les autres ne bougent pas — et tout le monde se trompe ensemble. On appelle ça le « pas de deux catastrophique » : deux agents qui, chacun raisonnant juste de son côté, produisent ensemble une régression. Rien ne garantit alors que l'équipe progresse.

## L'idée centrale

---

**On se corrige l'un après l'autre, pas tous à la fois.** Chaque agent s'ajuste *en tenant compte* du changement que les précédents viennent de faire. Cet ordre — d'apparence anodine — suffit à **garantir mathématiquement** que l'équipe ne recule jamais d'une étape à l'autre (amélioration dite *monotone*). C'est la grande force théorique de HAPPO.

## Le mécanisme, pas à pas

---

1. On fige une référence : la politique d'équipe actuelle.
2. **Agent 1** calcule sa meilleure petite correction (à la PPO, avec la laisse  $\varepsilon$ ) et l'applique.
3. **Agent 2** calcule la sienne, mais **en intégrant** ce que l'agent 1 a déjà changé.
4. **Agent 3**, puis 4, de même — chacun voit l'effet cumulé des précédents.
5. L'ordre peut être tiré au hasard à chaque tour, pour ne privilégier personne.
6. Comme chacun ne fait qu'un petit pas (laisse  $\varepsilon$ ) et tient compte des autres, le total des pas améliore l'équipe sans la déstabiliser.

## Les formules, traduites

---

### ① La mise à jour en cascade.

agent 1 se corrige → agent 2 se corrige (sachant le pas de 1) → agent 3 (sachant 1 et 2) →  
...

*Pour un littéraire* : des musiciens qui s'accordent **à tour de rôle**. S'ils tendaient tous leur corde en même temps sans s'écouter, l'orchestre sonnerait faux ; en s'accordant l'un après l'autre, chacun se règle sur ce que les précédents ont déjà fait.

### ② La garantie d'amélioration monotone.

valeur de l'équipe APRÈS le tour  $\geq$  valeur AVANT (jamais de recul, prouvé)

*Pourquoi c'est rare et précieux* : la plupart des méthodes multi-agents espèrent progresser ; HAPPO le **garantit**, à la manière dont TRPO/PPO le garantissaient pour un agent seul. La cascade est précisément ce qui transporte cette garantie du solo vers le groupe.

## Un exemple concret

Le chef décide de resserrer la formation ; **puis** le mitrailleur ajuste sa position de tir *en sachant* que la formation s'est resserrée ; **puis** le médecin se replace *en sachant* où sont désormais les deux autres. Personne n'optimise « dans le vide ». Si, au contraire, les trois s'étaient repositionnés simultanément en imaginant les autres immobiles, ils auraient pu se masquer mutuellement les angles de tir — une régression que la cascade évite.

## Variantes & pièges

- **HATRPO** : la même idée avec la contrainte dure de TRPO (plus rigoureuse, plus lourde).
- **HASAC** : variante hors-politique (entropie maximale), plus économe en données.
- **MAT** (*Multi-Agent Transformer*) : traite l'équipe comme une *séquence* à décider l'un après l'autre — cousin moderne de l'idée de cascade.
- *Pièges* : l'ordre séquentiel coûte un peu en calcul ; et il faut un critique central bien conçu, comme pour MAPPO.

## Pourquoi on l'utilise ici

Ton équipe est **hétérogène par conception**. HAPPO est taillé pour ça : il évite que les quatre se marchent dessus en se corrigeant en même temps, et il offre une **progression sûre** là où MAPPO « simple » suppose des agents semblables. C'est le moteur recommandé pour « 1 chef + 3 spécialistes ».

*En dernier regard. HAPPO énonce une vérité politique : une communauté de rôles distincts ne tient pas par la simultanéité, mais par l'ordre du dialogue. On ne parle pas tous en même temps ; on s'écoute, on tient compte de ce que l'autre vient de faire. Adam Smith y voyait la division du travail ; Durkheim, la « solidarité organique » — cette cohésion paradoxale où c'est la différence des fonctions, non leur uniformité, qui soude le corps social. L'algorithme retrouve l'intuition : quatre êtres dissemblables deviennent une équipe non en agissant pareil, mais en s'ajustant tour à tour, chacun à sa place et au su des autres.*

# QMIX & décomposition de valeur

---

« *Que l'intérêt de chacun serve le bien de tous* »

## Le problème qu'il résout

---

À l'exécution, chaque soldat décide **seul**, avec ses seuls yeux (pas de vue d'ensemble, pas de temps pour délibérer). Mais la récompense est **collective**. Comment garantir que « chacun fait au mieux pour soi, localement » revienne à « l'équipe fait au mieux, globalement » ? Si l'on apprend une note d'équipe géante (dépendant des actions de tous), elle est impossible à utiliser sur le terrain (chacun devrait connaître les actions des autres). Si chacun apprend dans son coin (*Independent Q-Learning*), la coordination s'effondre. Il faut un pont entre les deux.

## L'idée centrale

---

**Construire la note d'équipe à partir des notes individuelles, selon une règle qui force la coïncidence des optima.** Chaque agent a sa propre note ; une « note d'équipe » les combine ; et la combinaison est choisie de telle sorte que maximiser sa note individuelle revienne toujours à servir la note d'équipe. On apprend ensemble (avec la note d'équipe), on exécute séparément (chacun sa note).

## Le mécanisme, pas à pas

---

1. Chaque agent a un réseau qui lui donne une **note** (valeur) pour chacune de ses actions possibles, à partir de sa seule observation.
2. Un **réseau mélangeur** combine ces notes individuelles en une **note d'équipe**, en s'aidant de l'état global (disponible à l'entraînement).
3. On entraîne le tout pour que la note d'équipe prédise bien la récompense future réelle (apprentissage par valeur, type Q-learning, avec mémoire de rejou).
4. **Contrainte clé** : le mélangeur est forcé d'être *monotone* (voir ④) — c'est ce qui garantit la coïncidence des optima.
5. À l'exécution, on jette le mélangeur : chaque agent choisit l'action de **sa** meilleure note.

## Les formules, traduites

---

④ La règle de coïncidence (principe IGM).

meilleur choix de l'ÉQUIPE = (meilleur choix du soldat 1, ..., meilleur choix du soldat n)

*Pour un littéraire : on veut que la somme des décisions « les meilleures pour soi » forme exactement la meilleure décision collective. Que l'égoïsme local et le bien commun pointent dans la même direction. Sans cette propriété, la décentralisation trahit l'équipe.*

### ② La fabrication de la note d'équipe.

note d'équipe = mélange( note du soldat 1, ..., note du soldat n ; état global )

### ③ La règle de montée (monotonie).

si la note d'UN SEUL membre monte, la note d'équipe ne peut que monter ou rester égale

*Pourquoi cette règle garantit le ① : si améliorer un membre ne peut **jamais** nuire à l'équipe, alors chacun peut chercher son propre maximum sans risque pour le collectif. La monotonie est le pont entre l'individuel et le commun.*

④ **La hiérarchie d'expressivité.** VDN additionne simplement les notes (rudimentaire). QMIX les mélange de façon monotone mais **non linéaire**, en tenant compte de l'état (bien plus riche). QPLEX va plus loin encore (voir le piège).

## Un exemple concret

Le mitrailleur « note » très haut l'action *suppression* quand l'ennemi est groupé ; le voltigeur note haut *avancer* quand il est couvert. Le mélangeur apprend que, **ensemble**, ces deux notes hautes valent une excellente note d'équipe (suppression + progression = manœuvre réussie). Chacun, en suivant sa propre note, déclenche la bonne combinaison — sans s'être concerté.

## Variantes & pièges

- **La limite à connaître (capital).** Certaines manœuvres sont en **tout-ou-rien** : déborder à deux est excellent ; *un seul* qui s'élance sans couverture est *pire que rien*. Là, améliorer un seul membre **dégrade** l'ensemble — ce que la règle de montée **interdit de représenter**. QMIX est alors aveugle à ces tactiques.
- **VDN** (somme), **QTRAN** et **QPLEX** (qui lèvent la limite de monotonie, au prix d'un entraînement plus délicat), **Weighted QMIX** (corrige un biais).
- **Piège** : croire que QMIX peut tout représenter. Pour de la coordination « tout-ou-rien », il faut monter en gamme (QPLEX) ou passer à l'acteur-critique (MAPPO/HAPPO).

## Pourquoi on l'utilise ici

C'est la base de l'apprentissage **par valeur** en équipe : très économe en données (il ressasse une mémoire de jeu). On le garde en réserve pour des actions discrètes, en sachant sa limite — et on

lui préfère HAPPO quand la coordination devient « tout-ou-rien ».

*En dernier regard. QMIX pose, sans le dire, la plus vieille question de la métaphysique : le tout est-il la somme de ses parties ? Sa règle de montée est un pari d'harmonie — l'idée, déjà chère à Leibniz, que le bien de chacun puisse composer le bien de tous. Et sa limite est tout aussi profonde : il existe des tous qui sont \*plus\* que la somme — la manœuvre coordonnée, l'accord choral, l'élan collectif — où le sens naît de la relation, non de l'addition. L'algorithme trace ainsi, à sa manière, la frontière exacte entre ce que le réductionnisme sait reconstruire et ce qui lui demeure, irréductiblement, étranger.*

# COMA

« *Quel a été ton mérite, à toi, dans ce succès commun ?* »

## Le problème qu'il résout

La récompense est commune : l'équipe gagne ou perd ensemble. Mais alors, **comment savoir qui a contribué ?** Si l'on récompense tout le monde pareil après une victoire, le soldat resté à couvert sans rien faire reçoit le même crédit que celui qui a pris le risque décisif. Pire : son signal d'apprentissage lui dit « continue comme ça » — donc « reste planqué ». C'est le **fléau de l'assignation de crédit**, et il étouffe l'apprentissage : le bon signal de chacun est noyé dans le bruit des actions de tous.

## L'idée centrale

**Mesurer la contribution propre d'un agent en le comparant à lui-même.** On se demande : « *et si ce soldat avait fait autre chose, les trois autres agissant exactement de la même façon — le résultat aurait-il changé ?* » On isole ainsi ce que son geste, et lui seul, a apporté.

## Le mécanisme, pas à pas

1. Un **critique central** (qui voit tout, à l'entraînement) apprend à noter la situation pour *toutes* les combinaisons d'actions de l'équipe.
2. Pour évaluer l'agent *i*, on calcule la note réelle obtenue avec son action.
3. On calcule ensuite la note **moyenne** qu'on aurait eue s'il avait choisi *n'importe quelle autre* de ses actions — **en gardant les actions des autres figées**.
4. La différence entre les deux est son **avantage contrefactuel** : sa contribution nette.
5. On pousse sa politique dans le sens de cet avantage (à la manière d'un gradient de politique).

## Les formules, traduites

### ① L'avantage contrefactuel.

mérite du soldat *i* = (note de ce qui s'est passé) – (note moyenne s'il avait agi autrement, les autres inchangés)

*Pour un littéraire* : on rejoue mentalement la scène en ne changeant **que** ce soldat. S'il obtient à peu près le même résultat quoi qu'il fasse, c'est qu'il n'a rien apporté de décisif. Si son geste précis a fait basculer l'issue, son mérite est réel — et chiffré.

② **Ce qu'on neutralise, et pourquoi.** En gardant les autres figés, on **soustrait** tout ce qui ne dépendait pas de lui : la chance, l'action des coéquipiers, le contexte. Ce terme soustrait s'appelle

une *baseline* (un point de référence). Mathématiquement, le retirer **ne fausse pas** la direction de l'apprentissage, mais **réduit énormément le bruit** — c'est là toute l'astuce.

*En clair* : COMA calcule une **récompense de différence** — la différence que fait sa présence et son choix, et rien d'autre.

## Un exemple concret

L'équipe nettoie un bâtiment et réussit. Le médecin, lui, est resté en couverture à l'arrière. Question contrefactuelle : s'il avait fait autre chose (avancer, tirer), le résultat aurait-il changé ? Non — la pièce était déjà tenue. Son avantage est donc proche de zéro : pas de gros crédit, mais pas de blâme non plus. En revanche, le voltigeur qui a neutralisé le défenseur d'angle : sans son geste précis, l'assaut échouait. Avantage fortement positif → son comportement est renforcé.

## Variantes & pièges

- **Récompenses de différence** (Wolpert & Tumer, l'ancêtre conceptuel) : COMA en est la version neuronale apprise.
- **Lien avec la décomposition de valeur** (QMIX) : même but — démêler le crédit — mais côté *valeur* plutôt que côté *contrefactuel*.
- *Pièges* : le calcul exige un critère central précis ; et il suppose qu'on puisse énumérer les actions de l'agent (plus dur en actions continues).

## Pourquoi on l'utilise ici

Sans ce démêlage, le signal d'apprentissage d'équipe est inexploitable (« on a gagné, donc tout le monde a bien joué », même le passif). COMA fait remonter le **vrai** mérite de chacun, ce qui accélère l'apprentissage et **combat directement le « lazy agent »** (l'agent qui se planque) — un risque qu'on a justement voulu éviter dans ton modèle.

*En dernier regard. COMA repose sur l'intuition morale la plus tenace : la responsabilité se mesure au contrefactuel. « Aurais-tu pu agir autrement, et cela aurait-il changé quelque chose ? » — c'est la question des juges, des philosophes du libre arbitre, et de chacun à l'heure des regrets. L'algorithme porte d'ailleurs en germe la leçon du film « La vie est belle » : la valeur d'une existence se révèle en imaginant le monde sans elle. Attribuer un mérite, ici, ce n'est pas flatter ni punir ; c'est reconnaître, froidement et justement, la différence qu'un être a réellement faite.*

# CMDP / Lagrangien

« Réussir la mission sans franchir la ligne rouge »

## Le problème qu'il résout

Ton cahier des charges dit : « remplir la mission **et** rester en vie ». La tentation est de tout fondre dans une seule récompense : *réussite – pénalité\_pertes*. Mauvaise idée. D'abord, comment fixer le taux de change entre « un objectif pris » et « un homme perdu » ? Tout chiffre est arbitraire. Ensuite — résultat mathématique — une simple somme pondérée **ne peut atteindre que certaines solutions** (la partie « convexe » des compromis possibles) : des plans pourtant excellents deviennent **inatteignables** quel que soit le réglage. La survie n'est pas un objectif à marchander librement ; c'est une **limite**.

## L'idée centrale

**Maximiser la mission sous une contrainte de pertes**, et non additionner les deux. On ne cherche pas « un peu moins de morts contre un peu moins de mission » ; on cherche *la meilleure mission possible, tant que les pertes restent sous un seuil que tu fixes*. Pour résoudre ce problème « sous contrainte », on le transforme astucieusement en un problème ordinaire avec un **curseur auto-réglé**.

## Le mécanisme, pas à pas

1. On définit **deux** signaux : la **récompense** (réussite de mission) et le **coût** (les pertes).
2. On fixe un **seuil** de coût acceptable (« je n'accepte pas plus de tant de pertes attendues »).
3. On introduit un curseur  $\lambda$  (le « prix du risque »).
4. On entraîne la politique à maximiser : *réussite –  $\lambda \times$  (dépassement du seuil)*.
5. **On ajuste  $\lambda$  automatiquement** : si les pertes dépassent le seuil,  $\lambda$  monte (la prudence devient chère) ; si elles repassent dessous,  $\lambda$  redescend (l'audace revient). Ce va-et-vient converge vers le juste niveau.

## Les formules, traduites

### ① L'objectif sous contrainte.

maximiser (réussite de mission) TANT QUE (pertes attendues)  $\leq$  seuil que tu choisis

*Pour un littéraire : l'ordre de mission réaliste — « prends l'objectif, mais je n'accepte pas plus de tant de pertes ». Le seuil, c'est le niveau de risque acceptable.*

### ② Le curseur (le « lagrangien »).

$$\text{note} = (\text{réussite de mission}) - \lambda \cdot (\text{dépassement du seuil de pertes})$$

*Pourquoi ce tour de passe-passe marche* : transformer une contrainte en pénalité réglable est une technique classique d'optimisation (les « multiplicateurs de Lagrange »). Le génie ici est que  $\lambda$  n'est pas fixé à la main : il s'apprend.

### ③ Le réglage automatique de $\lambda$ .

trop de pertes  $\rightarrow \lambda$  monte (prudence chère) ; pertes sous le seuil  $\rightarrow \lambda$  baisse (audace permise)

*En clair* : tu ne règles pas un compromis figé ; tu poses une **limite**, et le curseur trouve de lui-même l'agressivité qui la respecte, en se resserrant ou se relâchant au fil de l'apprentissage.

## Un exemple concret

Au début, les agents foncent : pertes au-dessus du seuil.  $\lambda$  grimpe ; soudain, chaque mort « coûte » très cher dans la note, et la politique apprend à temporiser, à mieux se couvrir, à attendre la suppression avant d'avancer. Les pertes retombent sous le seuil ;  $\lambda$  se détend ; un peu d'audace revient. Le système oscille puis se stabilise sur le **plan le plus offensif compatible avec le seuil de pertes** — exactement ce qu'on voulait.

## Variantes & pièges

- **CPO** : impose la contrainte par une région de confiance (plus rigoureux).
- **MAPPO-Lagrangian / MACPO** : les versions **multi-agents sous contrainte** — celles de ton projet.
- **CVaR** (raffinement avancé) : au lieu de borner la *moyenne* des pertes, borner le *pire cas* (la queue de distribution) — plus réaliste pour du combat, mais plus exigeant.
- *Pièges* :  $\lambda$  qui oscille trop (réglage de sa vitesse d'ajustement) ; et la contrainte n'est garantie qu'à **convergence**, pas forcément pendant l'exploration (d'où le besoin de sécurité pendant l'entraînement).

## Pourquoi on l'utilise ici

C'est la traduction **exacte** de « remplir la mission et rester en vie », avec la survie posée comme contrainte d'équipe. Le curseur  $\lambda$  régule l'audace tactique tout seul — tu n'as qu'à fixer le seuil de pertes acceptable, le reste s'ajuste.

*En dernier regard. Ce curseur tranche une vieille querelle morale. Le conséquentialiste met tout dans une même balance et calcule le meilleur solde ; le déontologue pose des limites qu'aucun bénéfice ne saurait acheter. Le CMDP fait les deux à la fois : il maximise un résultat, mais sous une frontière qu'il traite comme inviolable. Et son  $\lambda$  — ce prix du risque qui s'élève à mesure qu'on approche de la limite — dit quelque chose de troublant et de juste : plus on s'approche de l'inacceptable, plus la prudence doit coûter cher, jusqu'à devenir prohibitive. Mettre un prix sur le risque sans jamais mettre un prix sur l'interdit : c'est, peut-être, une définition opératoire de la sagesse pratique.*

# Auto-jeu & PSRO / ligue

---

« *Se forger contre un adversaire qui apprend aussi* »

## Le problème qu'il résout

---

Une équipe entraînée contre un ennemi **scripté** (figé) devient experte... à battre cet ennemi-là. Face à un adversaire intelligent et imprévu, elle s'effondre : elle a appris des recettes contre un mannequin, pas l'art du combat. On dit qu'elle **sur-apprend** son adversaire. Il faut un opposant qui **progresses**, pour forcer l'équipe à progresser elle aussi.

## L'idée centrale

---

**Faire apprendre les deux camps l'un contre l'autre** — une course aux armements qui les hisse tous les deux. Mais avec une discipline : s'entraîner *seulement* contre sa version du moment fait **tourner en rond** (on oublie comment battre les anciennes tactiques) et peut même **cycler** sans fin (pierre-feuille-ciseaux). La théorie des jeux fournit la discipline qui évite ces pièges.

## Le mécanisme, pas à pas (du naïf au robuste)

---

1. **Auto-jeu naïf** : l'équipe s'entraîne contre une copie d'elle-même, mise à jour en continu. Simple, mais oublie et cycle.
2. **Jeu fictif** : l'équipe s'entraîne contre un **mélange de toutes ses versions passées** — elle reste bonne contre l'ancien comme contre le nouveau.
3. **PSRO** (la version disciplinée) : - on garde une **population** d'équipes (et d'adversaires) ; - on dresse la table « qui bat qui » en les faisant s'affronter ; - un **méta-solveur** (théorie des jeux : équilibre de Nash) calcule la **bonne dose** de chaque adversaire à affronter ; - on entraîne une **meilleure réponse** à ce mélange, qu'on **ajoute** à la population ; - on répète. La population s'enrichit de styles de plus en plus variés.
4. **Ligue** (façon AlphaStar) : on ajoute des agents aux rôles spécialisés — des « exploiters » dont **le seul but est de trouver les failles** de l'équipe principale, qui doit apprendre à les colmater (mécanisme dit *PFSP* : on affronte en priorité ceux qui nous mettent en difficulté).

## Les formules, traduites

---

### ① La meilleure réponse.

nouvelle équipe = la meilleure réponse possible au mélange d'adversaires du moment

*Pour un littéraire* : à chaque tour, on fabrique le meilleur « contre » à ce que joue l'autre camp.

### ② Le jeu fictif (pour ne pas oublier).

adversaire d'entraînement = un MÉLANGE des versions passées, pas seulement la dernière

*Pourquoi* : s'entraîner contre le seul présent fait perdre les parades du passé — et l'ennemi pourrait ressortir une vieille ruse. Le mélange immunise contre l'oubli.

### ③ La dose d'adversaires (méta-jeu).

on choisit COMBIEN affronter chaque style, selon la table « qui bat qui » (équilibre de Nash)

*En clair* : un tournoi permanent, où l'on choisit ses sparring-partners non au hasard, mais pour qu'ils corrigent précisément nos faiblesses du moment.

## Un exemple concret

Ton équipe bleue apprend à déborder par la droite. L'équipe rouge, qui apprend aussi, finit par **anticiper** ce débordement et poster un guetteur à droite. Bleu doit alors inventer la feinte (menacer à droite, frapper à gauche). Rouge réplique en gardant des réserves... À chaque tour, le niveau monte. La ligue ajoute un « exploiteur » qui ne fait *que* chercher la faille du moment (« bleu néglige ses arrières ») : bleu est forcé de la corriger, et devient robuste là où il était fragile.

## Variantes & pièges

- **NFSP,  $\alpha$ -PSRO, JPSRO** : variantes selon le concept de solution (Nash,  $\alpha$ -Rank, équilibres corrélés).
- *Pièges* : le **cyclage** (A bat B bat C bat A) si l'on ne garde pas l'historique ; l'**explosion** de la population (coûteux) ; et le piège du miroir — s'entraîner seulement contre soi peut produire une virtuosité qui tourne à vide, d'où la nécessité de la **diversité** (ligue, exploiters).

## Pourquoi on l'utilise ici

C'est le saut qualitatif vers des agents de combat **crédibles** : robustes face à des styles variés, et non sur-spécialisés contre un seul comportement scripté. On y vient *après* qu'une équipe sache déjà se battre contre un ennemi simple.

*En dernier regard. Il y a là une loi profonde de la formation de soi : on ne se forge que contre une résistance. Hegel en fit une dialectique — la conscience ne se révèle qu'au contact de ce qui lui résiste ; Nietzsche, une éthique de l'épreuve ; le judoka, un principe d'entraînement. Mais la méthode garde aussi un avertissement : s'affronter seulement soi-même mène au solipsisme, à une habileté tournant à vide. D'où la ligue, qui réintroduit l'altérité — des adversaires assez divers pour nous surprendre. On ne grandit ni dans le miroir seul, ni sans miroir ; on grandit face à d'autres, et c'est leur différence qui nous révèle nos angles morts.*

# PBT — Entraînement par population

---

« Une évolution darwinienne des entraînements »

## Le problème qu'il résout

---

Tout entraînement RL dépend d'une foule de **réglages** (vitesse d'apprentissage, dose d'exploration, poids divers...) dont on ne connaît pas d'avance les bonnes valeurs. La méthode habituelle — essayer une combinaison, attendre des heures, recommencer — est lente et coûteuse. Pire : les bons réglages **changent au cours** de l'entraînement (au début il faut explorer, plus tard il faut affiner). Un réglage fixe est donc forcément sous-optimal à un moment ou un autre.

## L'idée centrale

---

**Entraîner toute une population en parallèle, et laisser les bonnes recettes se répandre et muter, en continu.** Plutôt que de chercher les bons réglages d'avance, on les fait **évoluer** pendant l'entraînement : sélection des meilleurs, copie, petites mutations. La nature trouve à notre place.

## Le mécanisme, pas à pas

---

1. On lance **N entraînements en parallèle**, chacun avec des réglages un peu différents.
2. À intervalles réguliers, on **évalue** chaque membre (sa performance du moment).
3. **Exploiter** : un membre nettement moins bon **abandonne** son état et **copie** celui d'un membre nettement meilleur (poids du réseau et réglages).
4. **Explorer** : juste après avoir copié, il **perturbe** légèrement les réglages (au hasard, un peu) — autant de tentatives d'aller plus loin que le « parent ».
5. On laisse tourner. La qualité moyenne monte, et les réglages **s'adaptent dans le temps** sans qu'on touche à rien.

## Les formules, traduites

---

### ① Exploiter (sélection).

si un membre est nettement moins bon → il adopte l'état et les réglages du meilleur

*Pour un littéraire : les recettes perdantes sont abandonnées ; les gagnantes se reproduisent.*

### ② Explorer (mutation).

après avoir copié → on perturbe un peu les réglages (légèrement, au hasard)

*Pourquoi les deux ensemble* : copier seul fige la population sur un unique gagnant (on s'appauvrit) ; muter seul disperse sans capitaliser. **Sélection + variation** : c'est la formule même de l'évolution. La diversité de la population est une assurance contre l'impasse.

## Un exemple concret

Cinq entraînements de ton équipe tournent de front. Au bout d'une heure, trois stagnent, deux progressent vite (ils avaient, par chance, une meilleure dose d'exploration). Les trois retardataires **copient** l'un des deux bons, puis **perturbent** légèrement leurs réglages — l'un essaie un peu plus d'exploration, l'autre un peu moins. Une heure plus tard, c'est l'une de ces *mutations* qui mène, et les autres la copient à leur tour. Sans intervention, la population a trouvé un réglage que tu n'aurais pas deviné — et qui, en plus, **a changé** au bon moment.

## Variantes & pièges

- Se combine avec presque tout : PBT **par-dessus** MAPPO, ou **par-dessus** l'auto-jeu (c'est exactement ce que faisait AlphaStar : PBT + ligue).
- *Pièges* : il faut **plusieurs entraînements de front** (gourmand — mais ta machine s'y prête) ; une mauvaise mesure de « qui est le meilleur » fausse toute la sélection ; et la population peut **converger trop vite** vers un seul type si l'on n'entretient pas assez de diversité.

## Pourquoi on l'utilise ici

PBT **règle les réglages tout seul** (gros gain de temps et de performance) et **diversifie les stratégies** (assurance contre l'impasse). Il exploite à merveille ta workstation, capable de faire tourner plusieurs entraînements simultanés. On l'ajoute *en dernier*, comme enveloppe, une fois la base solide.

*En dernier regard. PBT incarne une épistémologie sans architecte : le progrès n'y vient pas d'un plan génial, mais de la variation aveugle suivie d'une sélection impitoyable. C'est l'idée de Darwin, que Popper transposa à la connaissance — nos théories, disait-il, « meurent à notre place » : on les laisse échouer pour ne pas échouer soi-même. Leçon d'humilité : on n'a pas besoin de savoir d'avance ce qui est juste, pourvu qu'on entretienne assez de diversité et qu'on accepte d'éliminer ce qui échoue. La sagesse n'est pas toujours dans le concepteur ; parfois, elle est répartie dans la population — et dans le temps qu'on lui laisse.*

# PLR — Curriculum automatique

---

« *Apprendre toujours au bord de sa compétence* »

## Le problème qu'il résout

---

Si tu entraînes ton équipe sur **une seule** mission, elle la **mémorise** sans rien comprendre de général : déplace l'objectif de cent mètres, change la position ennemie, et elle s'effondre. C'est le **sur-apprentissage de l'environnement**. La parade évidente — tirer des missions au hasard — est inefficace : la moitié sont trop faciles (l'équipe n'y apprend rien) ou trop dures (elle ne fait qu'échouer sans rien comprendre). On gaspille l'essentiel du temps d'entraînement.

## L'idée centrale

---

**Ne pas tirer les missions au hasard : rejouer en priorité celles où l'équipe a le plus à apprendre.** Ni trop faciles, ni impossibles — celles, justement, qui sont *en train* de s'apprendre. Et laisser ce choix se faire **tout seul**, à partir des erreurs de l'équipe.

## Le mécanisme, pas à pas

---

1. On dispose d'un vivier de missions (positions, cartes, dispositifs ennemis variés — éventuellement générés automatiquement).
2. Quand l'équipe joue une mission, on mesure son **potentiel d'apprentissage** : à quel point ses pronostics y sont encore démentis (l'ampleur de ses « surprises »).
3. On garde un **score de priorité** par mission.
4. Pour la suite, on **rejoue plus souvent** les missions à fort potentiel, **moins souvent** celles déjà maîtrisées (où l'équipe ne se trompe plus).
5. À mesure que l'équipe progresse, les priorités **se recalculent** : le programme se déplace tout seul vers la nouvelle frontière de difficulté.

## Les formules, traduites

---

### ① Le potentiel d'apprentissage d'une mission.

priorité d'une mission  $\propto$  ampleur des surprises qu'elle provoque (à quel point l'équipe s'y trompe encore)

*Pour un littéraire* : une mission est « intéressante » tant qu'elle nous surprend. Quand elle ne nous surprend plus, c'est qu'elle est maîtrisée : on passe à autre chose. Le symbole «  $\propto$  » se lit « proportionnel à ».

### ② Le tirage priorisé.

on rejoue souvent les missions à fort potentiel, rarement celles déjà acquises

*Pourquoi ça généralise* : en restant en permanence à la frontière, l'équipe est forcée de rencontrer sans cesse du *nouveau* difficile — donc d'apprendre des principes transférables, pas une carte par cœur.

## Un exemple concret

Au début, la mission « objectif à découvert, sans couverture » est ingérable : l'équipe échoue toujours, surprise maximale → forte priorité, on la rejoue souvent. Une fois la tactique de progression par bonds maîtrisée, cette mission cesse de surprendre → sa priorité chute, on la rejoue peu. Entre-temps, une mission « objectif en zone urbaine » devient la nouvelle frontière → sa priorité monte. Le « programme scolaire » de l'équipe s'écrit ainsi de lui-même, toujours calé sur ce qu'elle est mûre pour apprendre.

## Variantes & pièges

- **PAIRED / ACCEL** (conception non supervisée d'environnements) : un **générateur adverse** fabrique des missions *juste* assez dures (curriculum par « regret »).
- **Randomisation de domaine** : la version la plus simple (tout varier au hasard) — moins efficace, mais une bonne base.
- *Pièges* : mal mesurer le « potentiel » (on peut sur-rejouer des missions juste bruitées, pas réellement instructives) ; et oublier de garder un peu de missions faciles pour ne pas désapprendre.

## Pourquoi on l'utilise ici

C'est ce qui fait passer d'agents qui **mémorisent une carte** à des agents qui **généralisent** à des missions variées — la condition pour qu'ils soient compétents *au combat*, et pas seulement sur le terrain d'entraînement. Indispensable dès qu'on veut de la robustesse.

*En dernier regard. PLR redécouvre une vérité de toute pédagogie : on n'apprend que de ce qui résiste — mais pas trop. Vygotski l'appelait la « zone proximale de développement », cet étroit territoire entre l'ennui du déjà-su et le découragement de l'impossible, où se joue tout progrès réel. La machine y ajoute une élégance : c'est l'élève lui-même, par ses erreurs, qui désigne la prochaine leçon. Apprendre, ici, ce n'est pas subir un programme imposé, c'est revenir sans cesse là où le monde nous dément encore, et faire de cette résistance le moteur même de la croissance. Le bon maître, au fond, ne fait pas autre chose.*

# Le RL — ce qui fonctionne, ce qui résiste, et l'art de s'en servir

Maîtriser l'apprentissage par renforcement (RL), ce n'est pas connaître *plus* d'algorithmes. C'est savoir **quand il marche, pourquoi il échoue, et comment le rattraper**. Ce dossier est cette carte — en clair, sans bagage mathématique.

Rappel en une phrase : le RL apprend **par essai et erreur**, guidé par une simple **récompense**. On ne lui montre jamais la bonne réponse ; il la découvre. C'est sa force — et la source de tous ses ennuis.

## 1. Là où le RL excelle (ses terrains de prédilection)

Le RL est le bon outil quand **quatre conditions** sont réunies :

1. Il existe un **but mesurable** (une récompense), mais **pas de « bonnes réponses »** à montrer.
2. On peut **interagir énormément et à bas coût** (un simulateur, un jeu).
3. Le problème est **séquentiel** : chaque geste influence la suite.
4. On veut **optimiser fort** un objectif bien défini, parfois au-delà de l'humain.

Quand ces conditions sont là, les résultats sont spectaculaires :

| Réussite                                                          | Domaine                    |
|-------------------------------------------------------------------|----------------------------|
| DQN sur Atari, AlphaGo / AlphaZero                                | jeux                       |
| AlphaStar (StarCraft II), OpenAI Five (Dota)                      | jeux complexes, temps réel |
| Locomotion simulée (MuJoCo), ballons stratosphériques (Loon)      | contrôle                   |
| Placement de circuits (AlphaChip), refroidissement de datacenters | optimisation industrielle  |
| Ajustement des grands modèles de langage (RLHF)                   | l'IA qui te répond         |

*Le fil commun* : un objectif clair + une interaction massive et bon marché. **Ton projet coche les cases** (mission = récompense, Arma = simulateur).

## 2. Là où le RL résiste (les murs, en clair)

C'est la partie la plus importante : les **modes d'échec** du RL. Les connaître, c'est déjà à moitié les éviter.

① **La faim de données.** Le RL réclame des millions, parfois des milliards d'essais. Il échoue dès que l'interaction est rare, lente, chère ou dangereuse (un vrai robot, un vrai champ de bataille). *Image* : un apprenti qui n'apprend qu'en se trompant — combien de chutes faut-il, et peux-tu te les permettre ?

② **Le détournement de récompense.** L'agent optimise la *lettre* de la récompense, pas son *esprit*. Exemple célèbre : un bateau de course qui, au lieu de finir la course, tourne en rond pour ramasser indéfiniment des bonus. *Image* : tu paies au nombre de lignes de code écrites — tu obtiens du code verbeux, pas du bon code.

③ **La récompense rare.** Si le succès n'arrive presque jamais (« mission réussie » une fois sur mille), l'agent erre à l'aveugle, sans savoir quel geste, mille pas plus tôt, a compté. *Image* : apprendre à jouer aux échecs en n'apprenant que le résultat final, jamais la valeur d'un coup.

④ **L'exploration.** Comment découvrir un bon comportement qu'on n'a jamais vu ? Sans incitation à explorer, l'agent reste dans ce qu'il connaît. *Image* : on ne trouve pas une porte cachée si l'on ne cherche jamais derrière les murs.

⑤ **La fragilité.** Le RL est réputé capricieux : un simple changement de graine aléatoire ou d'un réglage peut donner deux résultats opposés. Deux entraînements « identiques », deux issues. *Image* : une recette qui réussit un jour et rate le lendemain, sans qu'on sache pourquoi.

⑥ **La cible mouvante (multi-agents).** Quand les autres agents apprennent aussi, le sol bouge sous les pieds de chacun : ce qui marchait hier ne marche plus. La propriété de stabilité que suppose le RL classique est rompue.

⑦ **Le sur-apprentissage de l'environnement.** L'agent maîtrise *la* carte d'entraînement et s'effondre sur une carte nouvelle. Il a mémorisé, pas compris. C'est le fameux « fossé simulation → réalité ».

⑧ **Pas de bon sens.** L'agent ne connaît *que* sa récompense. En dehors d'elle, aucune intuition, aucune morale, aucune prudence — sauf si tu les y as mises explicitement.

---

### 3. L'art : comment les experts font marcher le RL

---

À chaque mur, un (ou plusieurs) remède. **C'est ça, maîtriser le RL** : reconnaître le mur, et appliquer le bon remède.

| Le mur                       | Le remède                                                                                                                                                                      |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ① Faim de données            | Simulateur <b>rapide</b> ; mettre au point sur un <b>simulateur-jouet</b> d'abord ; réutiliser l'expérience ( <i>replay</i> ) ; apprentissage <b>par modèle</b> ; parallélisme |
| ② Détournement               | <b>Façonnage par potentiel uniquement</b> ; surveiller le comportement réel ; <b>apprendre</b> la récompense par imitation/IRL ; poser des <b>contraintes</b>                  |
| ③ Récompense rare            | <b>Curriculum</b> (du facile au dur) ; <b>motivation intrinsèque</b> (curiosité) ; réétiquetage des échecs (HER) ; <b>machines à récompense</b>                                |
| ④ Exploration                | Récompense de curiosité (RND) ; bruit structuré ; <b>auto-jeu</b>                                                                                                              |
| ⑤ Fragilité                  | Partir d'une <b>base solide</b> (PPO) ; régler soigneusement ; <b>plusieurs graines</b> ; suivi rigoureux (TensorBoard/W&B)                                                    |
| ⑥ Cible mouvante             | <b>CTDE</b> ; auto-jeu <b>discipliné</b> (jeu fictif, ligue)                                                                                                                   |
| ⑦ Sur-apprentissage          | <b>Randomisation</b> des scénarios ; <b>multi-missions</b> ; curriculum automatique (PLR)                                                                                      |
| ⑧ Pas de bon sens / sécurité | <b>CMDP / safe RL</b> ; contraintes dures ; supervision humaine                                                                                                                |

**Le réflexe maître** : ne jamais faire confiance aveuglément au comportement appris. Toujours **regarder** ce que l'agent fait réellement (en visuel, sur des cas), car c'est là qu'on attrape les détournements de récompense avant qu'ils ne pourrissent tout.

## 4. La question d'or : faut-il vraiment du RL ici ?

Un expert se la pose **avant** de coder. Le RL est souvent un **dernier recours**, pas un premier réflexe. Compare honnêtement :

| Si...                                                                                                              | ...préfère                                                      |
|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Tu as des <b>exemples</b> de bon comportement (démonstrations)                                                     | l' <b>imitation</b> (clonage, IRL) — bien plus simple et stable |
| Le problème a une <b>solution connue / calculable</b>                                                              | la <b>planification</b> ou l' <b>optimisation</b> classique     |
| Le comportement est <b>simple et descriptible</b>                                                                  | un <b>script / arbre de comportement</b> (BT)                   |
| Tu as une <b>récompense mais aucune étiquette</b> , c'est <b>séquentiel</b> , et tu peux <b>interagir beaucoup</b> | <b>le RL</b>                                                    |

Pour du combat, la réponse experte est rarement « tout en RL ». C'est un **hybride** : un arbre de comportement doctrinal pour le « comment » (se mettre à couvert, recharger), et le RL pour le « quand » et le « qui » (Module 7). On apprend ce qui est dur à scripter, on script ce qui est dur à apprendre.

## 5. Ce que tout cela signifie pour ton projet

Surprise utile : **notre plan complet n'est rien d'autre qu'un catalogue des remèdes ci-dessus, appliqués à ton cas.**

- Ta contrainte qui domine tout = **faim de données × lenteur d'Arma** → simulateur-jouet d'abord, abstraction des actions, hybride BT, amorçage par imitation. (Murs ①)
- Ta récompense de mission est **rare** et le sim est **riche** → machines à récompense + façonnage par potentiel **seulement**, pour éviter le détournement. (Murs ②③)
- Tu veux des agents **qui généralisent** → multi-missions, PLR. (Mur ⑦)
- Tu veux de la **stabilité** → MAPPO/HAPPO, plusieurs graines. (Murs ⑤⑥)
- Tu veux qu'ils **restent en vie** → CMDP. (Mur ⑧)

Autrement dit : **on n'improvise pas. On applique l'art.** Chaque choix de notre architecture répond à un mur connu du RL.

*En dernier regard. Le RL met à nu une vérité dérangeante : un agent ne devient que ce que sa récompense l'invite à devenir, et il poursuivra toujours la mesure plutôt que le sens. C'est la loi de Goodhart — « quand une mesure devient un objectif, elle cesse d'être une bonne mesure ». Tout l'art du RL tient dans cette lucidité : la récompense n'est jamais tout à fait le but ; elle n'en est qu'une ombre chiffrée. Maîtriser le RL, ce n'est donc pas dompter une machine, c'est cultiver une humilité — celle de qui sait l'écart irréductible entre ce qu'on peut mesurer et ce qu'on veut vraiment. Celui qui l'oublie fabrique des agents brillants et insensés ; celui qui s'en souvient fabrique des agents qu'on peut, enfin, laisser agir.*